Java compact cheatsheet

STEP 1 Complete Java Session

1.1 How Java Runs: Classes, Variables and Data Types

From source code to output A Java program goes through two stages. The compiler (javac) turns your .java source into bytecode in a .class file.

Classes and objects Every Java program lives inside a class : a blueprint that groups data (variables) and behaviour (methods). An object is a real thing built from that blueprint, with its own state.

Data types, sizes and default values Java has eight primitive types that hold the value directly, and non-primitive (reference) types such as String , arrays and every class, which hold a reference.

Variables and identifiers Identifier rules: only letters, digits, _ and $, must not start with a digit, and must not be one of Java's keywords (all keywords are lower case, such as class , int , static). Identifiers are case sensitive.

Widening, narrowing and overflow Arithmetic on byte , short and char is done in int . So this does not compile:

Static and non-static members A static member belongs to the class and exists once. A non-static (instance) member belongs to each object. A static method such as main has no object, so it cannot use instance members directly:

The main method

Access modifiers

Data types, sizes and default values -2^{b-1}

Narrowing (explicit) $200 - 256 = -56$

TRAP The default char is the null character `'\u0000'`, not a space. It prints as nothing visible, which is why it is often mistaken for a space.

EXAM Can a Java program run without a main method? A standalone application cannot: main is the entry point. Frameworks such as TestNG or JUnit provide their own main and call your annotated methods instead.

1.2 Operators, Conditions, Loops and Patterns

Operators and precedence

if, else if, nested if and switch Conditions must be boolean expressions built from relational and logical operators. Java does not treat 0 or an empty string as false, unlike Python and JavaScript.

while, do while, for and for each All three parts of a for are optional: for (; i < 5;) works if i is set before and changed inside. Leave out the condition and the loop never ends.

break, continue and labelled loops

Classic loop programs, and a prime bug The session's three prime checks get faster: count every factor (O(n)), stop at n/2, stop at √n. But they hide boundary bugs : none of them rejects 0 and 1, the n/2 version calls 4 prime, and the √n version calls 9 and 25 prime.

Pattern programs with nested loops A pattern is a grid: the outer loop chooses the row , the inner loop prints the columns of that row. The trick is the inner loop's range as a function of the row number.

Pattern programs with nested loops $n \times n$

TRAP Java's % keeps the sign of the left operand: `-7 % 3 = -1`. Python's % follows the right operand: `-7 % 3 = 2`. Programs that check "is odd" with `n % 2 == 1` fail for negative numbers in Java. Use `n % 2 != 0`.

EXAM This is boundary value analysis in practice. The edges here are 0, 1, 2, small composites such as 4, and perfect squares such as 9 and 25, exactly where < versus <= matters.

1.3 Methods and Constructors

Why methods A method is a named block of code that performs one task. Writing the code once and calling it many times removes repetition, makes the program easier to read, and means a fix in one place fixes every call.

Calling static and instance methods

Parameters, return values and pass by value Parameters are the variables in the signature, arguments are the values passed in the call. A method declared void returns nothing, any other return type must return a value of that type (Addition5 prints, Addition6 returns).

Method overloading Several methods may share a name if their parameter lists differ in number, type or order. The compiler picks the best match at compile time. A different return type alone is not enough.

The call stack and recursion Every call pushes a frame holding its parameters and local variables onto the call stack. Returning pops it. A method that calls itself is recursive , and it needs a base case, or the stack overflows with StackOverflowError .

Constructors

this and constructor chaining this refers to the current object. this.i = i separates the field from a parameter with the same name. this(...) calls another constructor of the same class and must be the first statement .

EXAM Printing an object without `toString()` shows something like `Cons9@1b6d3586`: the class name and a hash code, not the memory address itself. `c == c1` compares references, so two separate objects are never == even if their fields match.

1.4 OOP in Practice

The four pillars The word polymorphism joins the Greek poly (many) and morph (form). In code, it means the same call can behave differently depending on the arguments or on the object.

Overloading and method mapping Overloading is compile time polymorphism : the compiler fixes which method runs. When no overload matches the argument types exactly, it widens the arguments and picks the most specific overload that fits.

Inheritance and its types A child class extends a parent and gets its fields and methods without rewriting them. Every class without an extends silently extends Object . Inheritance models an is-a relationship: an IPhone is a SmartMobile, which is a Mobile.

Method overriding Overriding gives a child its own version of a parent method with the same name and the same parameters . Which version runs is decided at run time by the actual object, so it is called runtime polymorphism or dynamic binding.

super and the order of construction super refers to the parent part of the current object. super.method() calls the parent version of an overridden method, super.field reads a hidden parent field, and super(...) picks a parent constructor.

Upcasting, downcasting and instanceof The session's DownCasting program writes (Iphone5) new Mobile5() . It compiles, then fails at run time, because a plain Mobile5 object never becomes an iPhone.

Abstraction with abstract classes

Interfaces An interface is a contract: a list of methods a class promises to provide. A class can implement several interfaces, which is how Java offers multiple inheritance safely.

Encapsulation with getters and setters Encapsulation wraps data and the code that changes it in one class, and hides the data with private .

TRAP If two overloads fit equally well, for example `sum(int, long)` and `sum(long, int)` called with `sum(1, 2)`, the call is AMBIGUOUS and does not compile.

EXAM Testers meet encapsulation in the Page Object Model: WebElements are private fields of a page class, and tests call public methods such as `login(user, pass)` instead of touching locators directly.

1.5 Strings, Arrays, Collections and Exceptions

Strings, the pool and equals A String is an object holding a fixed sequence of characters. String literals are stored once in the String pool , so two variables with the same literal share one object. new String("java") always makes a separate object on the heap.

String methods

Classic string programs The session's string programs come up in almost every interview. Each one is a loop over characters or words plus one String method.

Arrays An array holds a fixed number of values of one type in consecutive slots. new int[5] fills every slot with the default 0, and the size never changes afterwards. Reading animals[2] is one step, O(1), whatever the size.

Wrapper classes Collections hold objects only, so List<Integer> stores wrappers. Java converts automatically: autoboxing wraps an int into an Integer, unboxing unwraps it. Testers use the parse methods constantly, because everything read from a web page is text.

The Collection framework

Exception handling An exception is an object describing a problem that stops normal flow, such as dividing by zero. Without handling, the program ends and prints a stack trace (the session's ExceptionSession7). With try and catch it recovers and carries on.

Checked and unchecked, throw and throws

Classic string programs $\frac{n^2}{2}$

TRAP Valid indexes run from 0 to `length - 1`. `nums[5]` on a 5 element array throws `ArrayIndexOutOfBoundsException` at run time. Note that `length` is a field for arrays and `length()` a method for Strings.

TRAP `Integer` objects from −128 to 127 are cached, so == works by luck for small values and fails at 128. Always compare wrappers with `equals()`.

EXAM The session's ExceptionSession6 catches an exception and throws `new RuntimeException("Denominator is zero")`. That wraps the problem in a clearer message, a pattern frameworks use to turn low level errors into readable test failures.

1.6 Reading Test Data Files

Why test data lives in files A data driven test keeps its inputs and expected results outside the code. Testers can add a row to a sheet without touching Java, the same test runs once per row, and secrets such as passwords stay out of the source.

Text files

Properties files for configuration Every value comes back as a String , so numbers need Integer.parseInt . Lines starting with # or ! are comments.

CSV rows

JSON with Jackson and JsonPath The session reads this file with JsonPath : JsonPath.parse(file).read("$.firstName") . $ is the root, .key goes into an object and [n] into an array.

Excel with Apache POI and Fillo The session uses Fillo , which lets you query a sheet like a database table: select username, password from sheet1 where status='Pass' . Fillo is built on Apache POI , the standard Java library for Office files.

From rows to a data driven test TestNG's @DataProvider returns Object[][] : one inner array per run, holding that run's arguments. Read any file into that shape and a single test method covers every row.

TRAP Java's `split(",")` drops TRAILING empty fields: `"user5,Psw5,".split(",")` has 2 parts, not 3. Pass a limit of −1, `split(",", -1)`, to keep them. Values that contain commas inside quotes need a real CSV parser such as OpenCSV, or Python's `csv` module.

EXAM Keep an EXPECTED column next to the inputs, as login.csv does. A data row without its expected result can only tell you that the code ran, not that it did the right thing.

1.7 Practice Program Bank

The whole list in one place Each row below is one program from the session's list. The link opens the chapter and block where it is solved. Programs marked "this page" are solved in the sections that follow, grouped so that one program handles several related tasks.

Arithmetic, swaps and characters 15 / 2 is 7 in Java because both operands are int. Cast one of them first, (double) a / b , to get 7.5. Casting the result instead, (double) (a / b) , gives 7.0, because the fraction was already lost.

Number programs A factor of n divides it with no remainder. Counting digits uses a do while loop so that 0 counts as one digit. The closed path program counts the loops in each digit: 0, 4, 6 and 9 have one, 8 has two, so 6784 → 1 + 0 + 2 + 1 = 4 .

Reverse, palindrome, Armstrong and neon Reversing uses n % 10 to take the last digit and n / 10 to drop it. Java's % keeps the sign, so −123 becomes −321 with no special case, and 120 becomes 21 because a leading zero has no value.

String programs Two items need a careful reading. "aabbbccccdddda" must print a:2 b:3 c:4 d:4 a:1 , so it counts runs of the same letter, not totals, which is why the final a stands alone.

Repeated characters and words "First repeating character" has two common meanings, and interviewers accept either if you say which one you chose. In "data driven tests" the first letter met for a second time while scanning is a , at index 3.

Array programs

Running totals and numbers beyond long The running total item starts at 5 and adds each value of [−4, 1, 3, 5] : 1, 2, 5, 10 , then adds those results: 1 + 2 + 5 + 10 = 18 .

Number programs $6784 \rightarrow 1 + 0 + 2 + 1 = 4$

Armstrong $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$

Running totals and numbers beyond long 1, 2, 5, 10

TRAP In JavaScript, `[10, 9, 1].sort()` gives `[1, 10, 9]`, because the default sort compares text. Always pass `(a, b) => a - b` for numbers.

EXAM Python's int has no size limit, and JavaScript has BigInt with literals such as `12n`. In Java, `Long.MAX_VALUE + 1` silently wraps to the most negative long, with no exception.

STEP 2 Java Basics

2.1 Language Fundamentals

Identifiers An identifier is any name you invent in a program. It can name a class, a method, a variable or a label.

Reserved words Some words already have a fixed meaning in Java, so you cannot use them as names. Classic Java has 53 reserved words : 50 keywords plus 3 reserved literals.

The 8 primitive data types Java is strongly typed . Every variable and every expression has a type, and the compiler checks every assignment. Apart from the 8 primitive types everything is an object.

Literals A literal is a fixed value written straight into the code. In int x = 10; the literal is 10 .

Widening and narrowing Java converts a smaller type into a bigger one automatically. This is called widening or implicit type casting, and it never loses information about the size of the number.

Arrays An array is an indexed group of a fixed number of values of the same type. Its biggest strength is reading or writing any slot instantly by index. Its biggest weakness is that the size can never change after creation.

Types of variables

Var-arg methods Before Java 1.5, a method had a fixed number of parameters. Need to add 2, 3 or 10 numbers and you wrote several methods. Var-arg methods accept any number of arguments, including zero.

The main() method The compiler does not care whether a class has a main() method. The JVM checks it at run time, and if it cannot find the exact form it stops with an error.

Command line arguments Arguments typed after the class name are passed to main as a String[] . They let you change a program's behaviour without changing its code.

Java coding standards Good names make code read like a story. Java has well known naming habits that every team expects.

The 8 primitive data types

range of an n-bit signed type = -2^{n-1} to $2^{n-1} - 1$

Literals $010_8 = 1 \times 8^1 + 0 \times 8^0 = 8$

Widening and narrowing

byte → short → int → long → float → double

EXAM Words that sound like keywords but are not: `main`, `String`, `length`, `delete`, `include`, `sizeof`, `then`, `exit`. In Java the keyword is `extends`, never `extend`, and `implements`, never `implement`.

TRAP Money and float do not mix. `0.1 + 0.2` is not exactly `0.3` in binary floating point. Use `BigDecimal` for money.

EXAM Floating point literals can only be written in decimal. `double d = 0x123.4;` is not allowed, but `double d = 0x123;` is fine because `0x123` is an int that gets widened.

EXAM `int[] a, b;` makes both arrays one dimensional. `int[] a[], b;` makes a two dimensional and b one dimensional. A dimension written before a name only works for the first name, so `int[] []a, []b;` does not compile.

EXAM An array's elements always get default values, even when the array is a local variable. `int[] a = new int[3];` inside a method gives three zeros. Only the variable a itself needs to be initialized.

EXAM `public static int main(String[] args)` compiles fine, but the JVM refuses to start it, because the return type must be `void`. Since Java 7, a class with only a static block and no main also refuses to run.

2.2 Operators and Assignments

Increment and decrement ++ adds 1 to a variable and -- takes 1 away. Each comes in two flavours.

Arithmetic operators The arithmetic operators are + - * / % . The type of the result follows one simple rule.

String concatenation The + operator is the only overloaded operator in Java. If at least one side is a String it joins text, otherwise it adds numbers. Java works from left to right , so the moment a String appears, every + after it joins text.

Relational operators < <= > >= compare numbers and chars. They work on every primitive type except boolean , and never on objects.

Equality operators == and != work on every primitive, including boolean , and also on object references.

The instanceof operator instanceof checks whether an object is of a certain type, which includes its parent classes and interfaces.

Bitwise and boolean operators The operators & | ^ work on whole numbers bit by bit, and on booleans as logic. ~ flips every bit of a whole number. ! flips a boolean.

Short-circuit operators && and || are the lazy cousins of & and | . They skip the right side when the left side already decides the answer.

Type casting

Assignment operators Every compound assignment hides a cast back to the type on the left. b += 1 means b = (byte)(b + 1) .

The conditional operator The only ternary operator in Java takes three parts: a condition, a value for true and a value for false. It is a compact if else that produces a value.

new, [ ] and the evaluation order new creates objects and [] declares or indexes arrays. Java has no delete operator, because the garbage collector removes unused objects.

Arithmetic operators
result type = the biggest of (int, type of a, type of b)

String concatenation
a + b + c + d = ((10 + 20) + 30) + "java" = 60 + "java" = "60java"

Bitwise and boolean operators
$4 = 100_2$, $5 = 101_2 \rightarrow 4 \,\&\, 5 = 100_2 = 4$, $4 \mid 5 = 101_2 = 5$, $4 \,\hat{}\, 5 = 001_2 = 1$

Type casting
$130 = 1000\ 0010_2 \rightarrow$ as a byte: $-128 + 2 = -126$

Assignment operators d /= 2 → d = 10

new, [ ] and the evaluation order
$1 + ((2 \times 3) / 4) \times 5 + 6 = 1 + (6 / 4) \times 5 + 6 = 1 + 1 \times 5 + 6 = 12$

TRAP Integer overflow is silent. `Integer.MAX_VALUE + 1` becomes `Integer.MIN_VALUE` without any error. Use `Math.addExact` or `long` when numbers can grow large.

EXAM Every comparison with `NaN` is false, even `NaN == NaN`. The only one that is true is `!=`. Use `Double.isNaN(x)` to test for it.

2.3 Flow Control

Three families of flow control Java runs statements from top to bottom unless something changes the order. Flow control statements are those somethings, and they come in three families.

if else In the last example b = false assigns false and then gives false, so the else part runs. Write b == false or simply !b .

switch When one value decides between many paths, switch is cleaner than a long chain of if else if .

while loop Use while when you do not know in advance how many times the body will run, for example reading lines until a file ends.

do while loop A do while loop checks its condition at the end, so the body always runs at least once . A menu that shows itself before asking "again?" is the classic use.

for loop The most used loop. Use it when you know how many times to repeat.

for-each loop Java 5 added the for-each loop, the easiest way to visit every element of an array or a collection.

break break jumps out of the nearest switch or loop immediately. It is also allowed inside a labeled block. Anywhere else it does not compile.

continue continue skips the rest of the current pass and moves on to the next one. It is allowed only inside loops.

Labeled break and continue In nested loops, a plain break or continue only affects the inner loop. Put a label on the outer loop to target it instead.

EXAM An `else` always belongs to the NEAREST unmatched `if`, whatever the indentation says. This is the "dangling else".

EXAM If a and b were `final`, then `a > b` would become the constant `false` and the loop body would be an unreachable statement.

EXAM In a `do while` loop, `continue` jumps to the condition check at the bottom, not to the top of the body. Trace the next example by hand before you look at the output.

STEP 3 Object Oriented Java

3.1 Declarations and Access Modifiers

Java source file structure

The import statement Without an import you must write the full name of a class every time: java.util.ArrayList<String> list = new java.util.ArrayList<>(); . An import lets you use the short name.

Static import Java 5 added static import , which lets you use static members without writing the class name.

Packages A package groups related classes and interfaces, the way folders group files. It solves name clashes, improves modularity and adds a layer of access control.

Class level modifiers A top level class may use only these modifiers: public , default (no word), final , abstract and strictfp . Inner classes may also be private , protected and static .

final JVM does not give final fields a default value. You must assign them yourself, exactly once, at one of these places:

abstract An abstract method says what a child must do, without saying how. It has no body and ends with a semicolon. An abstract class is a class that may contain such methods and cannot be instantiated.

public, protected, default and private These four control who can see a field, method or constructor. From most restricted to most open:

static, native, synchronized, transient, volatile, strictfp A static member belongs to the class, not to any object. It can be used through the class name without creating an object. Static code cannot touch instance members directly, because there may be no object at all.

Interfaces An interface is a contract: a list of services a class promises to provide. The person using it only needs the contract, not the implementation. JDBC is a classic example: Java defines interfaces like Connection , and each database vendor writes the classes.

Naming conflicts in interfaces If two interfaces declare the same method with the same signature and return type, one implementation in the class satisfies both. If they have the same signature but different return types, no class can implement both.

Marker interfaces and adapter classes A marker interface has no methods at all. Implementing it simply tags your class with an ability that the JVM or a library checks for.

Abstract class or interface

public, protected, default and private
private < default < protected < public

TRAP Static imports save typing but can hide where a name comes from. Use them sparingly, for things like `Math` functions or test assertions.

EXAM `abstract` never talks about implementation, so it clashes with modifiers that do: `final`, `static`, `private`, `synchronized`, `native` and `strictfp` (in older Java). `abstract final` is the most common trap: one says "must be overridden", the other says "can never be overridden".

EXAM "An abstract class cannot be instantiated, so why does it have a constructor?" Because creating a child object runs the parent constructor too, which initializes the fields the abstract class defines.

3.2 OOP Concepts

Data hiding Data hiding means outsiders cannot touch an object's data directly. They have to go through methods, and those methods can check every request. In Java you hide data by making fields private .

Abstraction Abstraction means showing what something does while hiding how it does it. An ATM shows "withdraw" and "check balance", and hides the network calls and database updates behind the screen.

Encapsulation Encapsulation wraps data and the code that works on it into one unit, a class. Every Java class is encapsulated. When it also hides its data (data hiding) and offers services instead (abstraction), we get the full benefit.

IS-A relationship (inheritance) The IS-A relationship is inheritance, written with extends . A Dog IS-A Animal , so it gets everything an Animal has. The big win is code reuse .

HAS-A relationship HAS-A means one object holds a reference to another. A Car HAS-A Engine . There is no special keyword, you just declare a field. It also reuses code, without the tight bond of inheritance.

Method signature A method's signature is its name plus the types of its parameters, in order. The return type is not part of it. The compiler uses signatures to find the right method.

Overloading Methods are overloaded when they share a name but have different parameter lists. The compiler picks one by looking at the argument types at compile time, so overloading is called compile-time or static polymorphism.

Overriding A child overrides a parent method by declaring the same signature with its own body. Which body runs is decided at run time , by the actual object. That is why overriding is called runtime polymorphism, dynamic method dispatch or late binding.

Overloading versus overriding

Polymorphism Polymorphism means "many forms": one name, different behaviour depending on the situation. Java has two kinds.

Static control flow When a class is loaded, Java runs its static parts in three steps:

Instance control flow Every time you create an object, Java runs these steps:

Constructors Creating an object is not enough. It also needs its starting values. A constructor runs automatically at creation to do exactly that.

Singleton classes A singleton class allows only one object. Runtime is one: Runtime.getRuntime() always returns the same object. Singletons save memory when every user can share one object, such as a configuration or a connection pool.

Coupling and cohesion

Object type casting Casting an object reference changes the type of the reference , never the object itself.

Encapsulation
Encapsulation = Data hiding + Abstraction

Coupling and cohesion
Good design = loose coupling + high cohesion

Object type casting A b = (C) d;

EXAM If the parent class is not tightly encapsulated, no child can be either, because the child inherits the non-private field.

EXAM Overload resolution looks at the REFERENCE type, never the runtime object. With `m(Animal)` and `m(Monkey)`, the call `m(a)` where `Animal a = new Monkey();` picks `m(Animal)`.

EXAM Constructors and instance blocks both run at object creation. The instance blocks run first, then the constructor body.

3.3 Inner Classes

Why inner classes exist Declare a class inside another class when the inner thing cannot exist without the outer thing. A university has departments, and a map has entries. Without a Map object there is no Map.Entry .

Normal inner classes A normal inner class is declared directly inside the outer class body. Compiling produces two files: Outer.class and Outer$Inner.class .

Method local inner classes A class declared inside a method exists only inside that method. Use it for a helper that is needed again and again inside one method and nowhere else.

Anonymous inner classes An anonymous class has no name. You declare it and create its one object in the same expression. It is perfect for a quick, one time change of behaviour.

Normal class versus anonymous class

Static nested classes A nested class declared static is not tied to any outer object. Create it with new Outer.Nested() .

Normal inner versus static nested

Classes and interfaces inside each other

EXAM A method added only in the anonymous class cannot be called through the parent reference, because the compiler only knows the parent type.

STEP 4 Core Java APIs

4.1 java.lang: Object, String and Wrappers

The Object class java.lang.Object is the root of every class. If a class extends nothing, it extends Object directly. So every object you will ever create has Object's methods.

toString() println(obj) calls obj.toString() for you. Object's version returns the class name, an @ and the hash code in hexadecimal, which is rarely what a human wants.

hashCode() Hash based collections such as HashMap and HashSet use the hash code to decide which bucket an object goes into, so they can find it again quickly. A good hash code spreads objects evenly.

equals() and == Object's equals does the same as == : it only checks whether both references point at the same object. Override it to compare contents .

clone(): shallow and deep copies clone() makes a copy of an object. The class must implement the marker interface Cloneable , or Object's clone throws CloneNotSupportedException .

getClass() getClass() returns a Class object describing the real class of an object at run time. It is the entry point to reflection.

String is immutable Once a String object is created, its contents can never change. Every method that seems to change it actually returns a new String.

The string constant pool String literals are stored in a special area called the string constant pool (SCP). Before creating a literal, the JVM checks the pool. If an equal String is already there, it is reused.

Important String methods

StringBuffer When the contents change often, String wastes objects. StringBuffer is a mutable sequence of characters: every change happens in the same object.

StringBuilder and choosing between the three StringBuilder (Java 5) has exactly the same methods as StringBuffer, but none of them are synchronized. With one thread, which is almost always the case, it is faster.

Wrapper classes Each primitive type has a wrapper class that holds the value as an object: Byte Short Integer Long Float Double Character Boolean . Collections and generics only store objects, so wrappers are everywhere.

Autoboxing and unboxing Since Java 5 the compiler converts between primitives and wrappers automatically.

final versus immutable

hashCode()
hash("Ab") = 'A' × 31 + 'b' = 65 × 31 + 98 = 2113

StringBuffer
new capacity = (old capacity + 1) × 2

EXAM How many objects does `String s = new String("spring");` create, if the pool is empty? Two: one on the heap and one literal in the pool. Run it a second time and only one more appears, because the literal is reused.

TRAP Never compare wrapper objects with ==. Use `equals`, or unbox them first.

4.2 Exception Handling

What is an exception An exception is an unwanted, unexpected event that disturbs the normal flow of a program: a file that is missing, a division by zero, a number typed as text.

The runtime stack and default handling The JVM gives every thread its own runtime stack . Each method call pushes a stack frame , and each return pops it.

The exception hierarchy Throwable is the root of everything that can be thrown. It has two children.

Checked and unchecked exceptions The compiler checks some exceptions to make sure you handle them. These are checked exceptions. The rest are unchecked .

try and catch Put the risky code in try and the handling code in catch .

Printing exception information

finally Clean-up code, such as closing a file or a database connection, must run whether things went well or not. Put it in finally . It runs after the try, with or without an exception, caught or not.

final, finally and finalize

Nested try catch finally A try block can contain another try. The inner catch handles what it can, and anything else bubbles to the outer catch. Both finally blocks always run, inner first.

throw Sometimes you create an exception object yourself and hand it to the JVM. That is what throw does.

throws If a method can raise a checked exception and does not want to handle it, it must say so with throws . The responsibility moves to the caller.

Custom exceptions Your own problems deserve their own names, like InsufficientFundsException or TooYoungException . Create them by extending an existing exception class.

The top ten exceptions Exceptions are raised either by the JVM automatically, or by a programmer or API on purpose.

Modern exception handling Resources declared in the brackets after try are closed automatically, in reverse order, even if an exception happens. No finally needed. Any class that implements AutoCloseable works.

EXAM Whether an exception is checked or unchecked is decided by the compiler, but EVERY exception happens at run time. "Checked" never means "happens at compile time".

TRAP Returning from finally hides exceptions and confuses readers. Use finally for clean-up only.

EXAM The only thing that stops finally is the JVM itself shutting down, for example `System.exit(0)`.

4.3 Enum

Why enums An enum is a type with a fixed list of named values. It arrived in Java 5.

Declaring and using an enum

Enums in switch Enums have been allowed in switch since Java 5. Inside the cases, write the constant name without the enum name.

java.lang.Enum Every enum is automatically a child of java.lang.Enum . Since a class can extend only one class, an enum cannot extend anything else, and nobody can extend an enum.

values(), ordinal() and valueOf()

Constructors, fields and methods An enum can hold data. Give it fields and a constructor, and pass values in brackets after each constant.

Constant-specific bodies Each constant can override a method with its own body. Declare the method abstract in the enum and every constant must provide it.

An enum with main An enum is a class, so it can have a main method and be run directly. Remember that constants must come first.

enum, Enum and Enumeration

EnumSet and EnumMap Java has collections built specially for enums. They are faster and smaller than general purpose ones, because they use the ordinal internally.

TRAP Do not store ordinals in a database. Adding a constant in the middle changes every number after it. Store the name instead.

4.4 Regular Expressions

What is a regular expression A regular expression (regex) describes a group of strings that follow one pattern. For example, [6-9][0-9]{9} describes every Indian mobile number.

Pattern and Matcher Java's regex classes live in java.util.regex .

Character classes Square brackets match one character from a set.

Predefined character classes

Quantifiers Quantifiers say how many times something may repeat.

Splitting text Both Pattern and String can split text around a regex.

Validating input matches() checks whether the whole string fits the pattern, unlike find() , which looks for a piece of it.

Find, extract and replace Round brackets create capturing groups , which you can read with group(1) , group(2) and so on, or reuse in a replacement as $1 .

Validating input [a-k][a-zA-Z0-9#]+

EXAM A literal dot must be escaped: "`\\.`". Without the backslash, `.` matches every character, which is a classic bug when splitting on dots.

TRAP Real email rules are very complicated. For production, use a well tested library and confirm the address by sending a mail.

4.5 Internationalization (I18N)

What is I18N Internationalization (shortened to I18N, because there are 18 letters between the I and the N) means designing a program so it can adapt to many countries: their number formats, currencies, dates and languages.

Locale A java.util.Locale object describes a language and, optionally, a country. It is final , Cloneable and Serializable .

NumberFormat java.text.NumberFormat is abstract. Ask its factory methods for an object, then call format to turn a number into text, or parse to go the other way.

Controlling digits A NumberFormat can be told how many digits to show before and after the decimal point.

DateFormat java.text.DateFormat formats an old style java.util.Date . Pick a style and a locale.

Modern dates with java.time

ResourceBundle: translated messages Text shown to users should never be hard coded. Put each language's messages in its own resource bundle and let Java pick the right one for the locale. Bundles are usually .properties files, such as Messages_hi.properties .

TRAP `parse` throws the checked `ParseException` when the text does not fit the format, so it must be handled or declared.

EXAM NumberFormat rounds with HALF_EVEN (banker's rounding) by default. So with 0 decimals, 2.5 becomes 2 and 3.5 becomes 4.

STEP 5 Collections and Generics

5.1 Collections Framework

Arrays versus collections Arrays have three big limits: their size is fixed, they hold only one type, and they come with almost no ready-made methods. Collections fix all three.

The nine key interfaces add , addAll , remove , removeAll , retainAll , clear , contains , containsAll , isEmpty , size , toArray and iterator . Collection has no method to get an element by position, because not every collection has positions.

The List interface A List keeps elements in insertion order, gives every element an index, and allows duplicates and nulls.

ArrayList ArrayList is a resizable array . It is the most used collection in Java.

LinkedList LinkedList is a doubly linked list : each element knows the one before it and the one after it. It implements both List and Deque .

Vector and Stack Vector (Java 1.0) is an older resizable array whose methods are all synchronized. Stack extends Vector and works last in, first out . Both are called legacy classes.

Cursors: Enumeration, Iterator, ListIterator A cursor walks through a collection one element at a time.

Set, HashSet, LinkedHashSet and TreeSet A Set never holds duplicates. Adding one again simply returns false .

Comparable and Comparator Sorting needs a way to compare two objects. Java offers two.

Map and its classes A Map stores key → value pairs. Keys are unique, values may repeat. Each pair is a Map.Entry . Map is not a child of Collection.

NavigableSet and NavigableMap Java 6 added navigation methods to TreeSet and TreeMap, answering questions like "the closest value at or below this one".

Queue and PriorityQueue A Queue holds elements waiting to be processed, usually first in, first out.

The Collections and Arrays classes binarySearch needs a sorted list. If the key is missing it returns −(insertion point) − 1 . Above, "J" would go at index 1, so the result is −1 − 1 = −2.

Modern collection tools

EXAM With a `List<Integer>`, `remove(1)` removes the element at INDEX 1. To remove the value 1, write `remove(Integer.valueOf(1))`.

TRAP Changing a collection directly while a for-each loop or iterator walks over it throws `ConcurrentModificationException`. Iterators are FAIL-FAST . Use the iterator's own `remove`, or `removeIf`.

EXAM If a TreeSet's comparator returns 0 for two different objects, the set treats them as duplicates and keeps only the first. Above, "kiwi" and "date" have the same length, which is why the second comparison step matters.

5.2 Generics

Why generics Generics (Java 5) solve two problems that old collections had.

Generic classes A generic class declares a type parameter in angle brackets. The caller fills it in.

Bounded types Sometimes only certain types make sense. <T extends Number> allows Number or any of its children, and lets you call Number methods on T.

Generic methods A single method can be generic too. Declare the type parameter just before the return type .

Wildcards: ?, ? extends and ? super A wildcard ? stands for "some type I do not know". It is used in parameters and variables, not when creating objects.

Generics and polymorphism

Type erasure Generics exist only for the compiler. After checking, it erases the type arguments, so at run time an ArrayList<String> and an ArrayList<Integer> are the same class.

Raw types and legacy code A generic class used without type arguments, like plain ArrayList , is a raw type . Raw types exist so that pre Java 5 code still compiles, and the compiler only warns about them.

TRAP Never ignore "unchecked" warnings. Wrap risky legacy calls, or check the contents before trusting them.

STEP 6 Multithreading

6.1 Multithreading

Multitasking Multitasking means doing several tasks at the same time. It comes in two flavours.

Defining a thread by extending Thread Which line prints first is decided by the thread scheduler , a part of the JVM and operating system. Its algorithm is not fixed, so the exact order can change from run to run.

Defining a thread with Runnable The second way: implement Runnable (a functional interface with one method, run()), and hand it to a Thread .

Names and priorities Every thread has a name: one you give, or a default like Thread-0 . The thread that runs main is called main .

The thread life cycle

yield, join and sleep

Interrupting a thread One thread can interrupt another that is sleeping or waiting by calling t.interrupt() . The sleeping thread wakes up with an InterruptedException . If the target is not sleeping yet, the request is remembered and fires the next time it sleeps or waits.

Synchronization When several threads change the same data at once, updates can be lost. This is a race condition . count++ looks like one step but is three: read, add, write.

Inter-thread communication: wait and notify Threads can signal each other. A thread calls wait() on an object to pause until another thread calls notify() or notifyAll() on the same object.

Deadlock A deadlock happens when two threads wait for each other forever. Thread 1 holds lock A and wants lock B. Thread 2 holds lock B and wants lock A. Neither can move.

Daemon threads A daemon thread runs in the background to support other threads. The garbage collector is the famous example. When the last non-daemon thread ends, the JVM stops all daemon threads and exits.

Modern concurrency Creating threads by hand is fine for learning. Real programs use the java.util.concurrent package.

EXAM A thread can be started only once. Calling `start()` a second time throws `IllegalThreadStateException`.

TRAP Priorities are only hints. Many operating systems ignore them, so never rely on priority for correctness.

STEP 7 Files and Serialization

7.1 File I/O

The File class A java.io.File object represents a path to a file or folder. Creating the object does not create anything on disk.

FileWriter FileWriter writes characters to a file. By default it overwrites the file. Pass true as a second argument to append instead.

FileReader FileReader reads characters. read() returns the next character as an int , or −1 at the end of the file.

BufferedWriter and BufferedReader Buffered classes collect characters in memory and move them in big chunks, which is much faster. They also understand lines .

PrintWriter PrintWriter is the most convenient writer. It can wrap a file name, a File or another Writer, and its print , println and printf accept any type, just like System.out .

Classic file programs These small programs combine the classes above. They are common interview and lab exercises.

Byte streams and character streams

Modern file handling with java.nio

TRAP Forgetting `close()` (or `flush()`) can leave the last characters unwritten. Use try-with-resources so files always close.

EXAM `write(100)` writes the character d. `print(100)` writes the three characters `100`.

7.2 Serialization

What serialization means An object lives in memory. When the program ends, it is gone. Serialization is the process of writing an object's state as bytes, so it can be stored in a file or sent over a network.

Writing and reading an object The loaded object has the same state , but it is a brand new object, so p == q is false.

The transient keyword Some fields should never be saved, such as a password or a value that only makes sense in this run. Mark them transient . The JVM skips them while serializing and gives them their default value (0, false or null) after deserialization.

Object graphs When an object is serialized, every object it refers to is serialized too, and every object those refer to, and so on. This set of objects is called the object graph . Every object in the graph must be Serializable.

Customised serialization Marking a password transient loses it completely. If you want to save it in a protected form, add two private methods with these exact signatures. The JVM calls them automatically during serialization and deserialization.

Serialization and inheritance Then every child is Serializable automatically, even without writing implements Serializable . This is why many library classes are Serializable: a parent such as Number already is.

Externalizable Serializable saves the whole object automatically. Externalizable hands complete control to you: you decide which fields to save and how. It extends Serializable and adds two methods.

serialVersionUID Every Serializable class has a version number called serialVersionUID . It is written into the bytes with each object. When the bytes are read, the JVM compares that number with the one in the class it has now.

Modern advice For data shared with other programs or other languages, most teams prefer text formats such as JSON or compact formats such as Protocol Buffers.

EXAM The exam favourite: `transient static` and `transient final` both compile, and in both cases `transient` has no effect.

TRAP Reversing a string is only a teaching example, not real protection. Real systems never store passwords in a serialized file. They store a salted hash in a secure place.

EXAM Writing succeeds, reading fails with `InvalidClassException`. Every non-serializable parent must have an accessible no-argument constructor.

TRAP Never deserialize bytes from a source you do not trust. Crafted input can run unexpected code during loading. Since Java 9, an `ObjectInputFilter` can limit which classes are accepted.

STEP 8 JVM and Tooling

8.1 Garbage Collection

Why a garbage collector Every object you create with new lives in an area of memory called the heap . Memory is limited, so space taken by objects nobody uses any more must be given back.

Four ways to make an object eligible To watch an object without keeping it alive, these examples use a WeakReference . Its get() method returns the object while it exists, and null after the collector has removed it.

Requesting a collection You can ask the JVM to run the garbage collector in two ways. Both do the same thing.

Looking at memory with Runtime This program was started with java -Xmx64m Main , so the maximum heap is 64 MB. One million int values take 1,000,000 × 4 bytes ≈ 4 MB .

finalize and Cleaner Early Java let a class override finalize() , a method the collector would call once before removing an object. It is now deprecated for removal (since Java 18), because it is slow, unpredictable and can bring dead objects back to life.

Modern collectors and good habits The JVM ships with several collectors. You choose one with a command line flag. Their behaviour depends on the JVM vendor and version, which is why the language rules only promise when an object becomes eligible , not when it is collected.

Looking at memory with Runtime

$$\text{used memory} = \text{totalMemory} - \text{freeMemory}$$

EXAM A cycle of references does not keep objects alive. If none of them is reachable from outside, all of them are eligible.

TRAP It is only a request. The JVM may run a collection, delay it or ignore it. A JVM started with `-XX:+DisableExplicitGC` ignores every such call.

TRAP The cleanup action must not refer to the object it watches. If it did, the object would stay reachable and never be cleaned.

EXAM Classic facts that still appear in exams: the collector calls finalize at most once per object, any exception thrown by finalize during collection is ignored, and calling `obj.finalize()` yourself is just a normal method call that does not free memory.

8.2 Assertions

Why assertions A quick way to debug is to add System.out.println lines and remove them later. Forget one, and it prints in front of real users. Assertions solve this. You write a check once and leave it in the code. During development you run with assertions enabled .

assert as a keyword Before Java 1.4, assert was an ordinary identifier, so old code could name a variable or method assert . Since 1.4 it is a reserved keyword, and such code no longer compiles.

The two forms If assertions are enabled and the condition is false, the JVM throws AssertionError . If the condition is true, nothing happens.

Enabling and disabling Assertions are controlled when you start the program, not when you compile it.

Where to use them and where not The author believed turns % 4 is always 0 to 3. The assertion proves otherwise: in Java, the remainder of a negative number is negative. Math.floorMod(turns, 4) is the fix.

AssertionError AssertionError is a child of Error , so it is unchecked. Catching it is legal, but it is bad practice: an assertion failure means the program is in a state its author believed impossible.

EXAM Old exam questions ask what happens when you compile this with very old `-source` settings. Modern compilers no longer accept source levels that old, so in practice `assert` is always a keyword.

TRAP With `-ea` this prints `[b]`. Without it, `[a, b]`. Never put work the program needs inside an assertion.

8.3 Development Tools

Compiling with javac javac is the Java compiler. It reads .java source files and writes .class files containing bytecode, which any JVM can run.

Running with java The java command starts a JVM, loads the class you name, and calls its main method. A class in a package must be given by its fully qualified name .

The classpath The classpath is the list of places where javac and java look for classes: folders and JAR files. It can be set in three ways.

JAR files A JAR (Java ARchive) packs many class files and resources into one file. It uses the zip format. Libraries are shared as JARs, and a whole application can be one JAR.

System properties A system property is a name and value pair available to the whole program. The JVM sets many of them, and you can add your own with -D on the command line, before the class name.

JDK, JRE and JVM JDK = JRE + development tools and JRE = JVM + library classes

Modern shortcuts For a small program you can skip javac. The launcher compiles the file in memory and runs it.

> **JDK, JRE and JVM**
> JDK = JRE + development tools

EXAM The four cases above are a favourite exam pattern: which command compiles, which runs, and which error each mistake produces.

TRAP The manifest file must end with a newline. Without it, the last line, often the Main-Class line, is silently ignored.

EXAM `-D` options go BEFORE the class name. Anything after the class name becomes a command line argument in `args` instead.

STEP 9 Modern Java

9.1 Lambdas, functional interfaces and method references

Why lambdas exist Before Java 8, passing a small piece of behaviour (how to compare, what to do on a click, which items to keep) meant writing a whole anonymous class. Most of those lines were ceremony.

Lambda syntax A lambda is (parameters) -> body . The compiler works out the parameter types from the context, so you rarely write them.

Functional interfaces A functional interface is an interface with exactly one abstract method . A lambda is simply a short way to create an object of such an interface: the lambda body becomes that method.

Method references When a lambda only calls one existing method, a method reference says the same thing even shorter, with :: . s -> s.toUpperCase() becomes String::toUpperCase .

Composing functions Functional interfaces come with default methods that join small pieces into bigger ones, like Lego bricks. Predicate has and , or and negate . Function has andThen and compose . Comparator has comparing , thenComparing and reversed .

Capturing variables and this A lambda can read local variables from the code around it, but only if they are effectively final : assigned once and never changed.

Lambdas and checked exceptions The methods of Function , Consumer and friends do not declare throws , so a lambda passed to them cannot throw a checked exception such as IOException .

How a lambda runs inside the JVM An anonymous class becomes its own .class file (Main$1.class). A lambda does not. javac turns its body into a private method and leaves a single invokedynamic instruction where the lambda was. The first time that line runs, the JVM builds the object.

Real world scenario: a retry helper The helper does not know what it is retrying. The work arrives as a Supplier , and the meaning of "success" arrives as a Predicate . The same helper now retries API calls, database checks or UI conditions, and it reports every attempt.

TRAP Do not hide exceptions with an empty catch inside a lambda. A test that swallows an IOException will pass with wrong data, which is worse than a clear failure.

9.2 Streams, Collectors, Optional and Gatherers

What a stream is A stream is a pipeline that data flows through. You describe the steps (keep these, change them to that, stop after three), and Java runs them for you. A stream does not store data.

Laziness and short circuits Intermediate operations are lazy . Building the pipeline does nothing. When the terminal operation asks for a result, elements travel through all the steps one at a time , and the work stops as soon as the answer is known.

Creating streams Almost anything can feed a stream: collections, arrays, number ranges, single values, generators, the characters of a string and the lines of a file (Files.lines , covered in the file chapter).

Intermediate operations

Terminal operations A terminal operation runs the pipeline and returns something that is not a stream: a number, a boolean, an Optional, a collection, or nothing at all (forEach).

Collectors: grouping and summarising collect with the helpers in Collectors answers report style questions in one line: how many per status, average time per suite, names of the slow tests.

Primitive streams Stream<Integer> stores every number as an object. IntStream , LongStream and DoubleStream work with plain numbers, so they are faster and add sum , average and summaryStatistics . Use mapToInt to enter them and boxed to leave.

Optional Optional<T> is a box that holds one value or nothing. A method that returns Optional tells the caller, in its signature, "I might not find anything". It replaces returning null and the NullPointerException that follows.

Stream gatherers (Java 24) Before Java 24, if no built in intermediate operation fitted, you had to leave the stream. Gatherers (JEP 485, final in Java 24, so in the Java 25 LTS) are custom intermediate operations, plugged in with gather(...) . The JDK ships useful ones in Gatherers .

Parallel streams parallel() splits the work across the CPU cores using the common ForkJoinPool. It helps for large amounts of pure calculation with no shared state. It hurts for small lists, for I/O, and for anything that writes to a shared object.

Real world scenario: a nightly test report A test that both passed and failed across runs is flaky : groupingBy plus mapping to a Set of statuses finds them in one pass. The same few lines work on real data read from JUnit XML or a database.

TRAP `orElse(x)` always evaluates x, even when a value is present. If the default is expensive (a database call, creating a user), use `orElseGet(() -> ...)`, which runs only when the box is empty.

9.3 Records, sealed types and pattern

matching

var: let the compiler write the type var (Java 10) asks the compiler to work out the type of a local variable from the value on the right. The variable still has one fixed type. It is not like JavaScript, where a variable can hold anything.

Text blocks A text block (Java 15) is a string that spans several lines between triple quotes. Indentation shared by all lines is removed, line breaks are kept, and double quotes need no escaping. It is perfect for JSON request bodies, SQL and expected HTML in tests.

Switch expressions The old switch statement falls through to the next case unless you remember break . That single missing word has caused many real bugs.

Records A record (Java 16) is a class whose job is to carry data. From one line, Java writes the private final fields, the constructor, an accessor per field (named name() , not getName()), and equals , hashCode and toString based on all fields.

Sealed classes and interfaces A sealed type (Java 17) lists exactly which classes may extend or implement it, using permits . It is the middle ground between final (nobody) and a normal class (anybody).

Pattern matching for instanceof The old pattern was test, then cast, then use. Since Java 16, o instanceof String s does all three: if the test passes, s is a String variable ready to use. The variable is only in scope where the test is known to be true, a rule called flow scoping .

Pattern matching for switch Since Java 21, a switch can test the type of a value, add a condition with when (a guard), and even handle null as a case. Cases are checked from top to bottom and the first match wins.

Record patterns and unnamed variables A record pattern (Java 21) matches a record and pulls its fields out into variables in the same step, and patterns can be nested. Combined with sealed types and switch, this style is called data oriented programming .

Primitive patterns (preview in Java 27) Patterns so far only worked with objects. Primitive types in patterns, instanceof and switch (JEP 532, fifth preview in Java 27) lets you write case int c when c >= 500 and ask big instanceof int , which is true only when the value fits into an int without ...

Real world scenario: modelling API outcomes Every possible outcome is now a record that implements one sealed interface. The report switch must handle them all, guards pick out special cases such as a zero total or a 503, and nested record patterns read the order fields directly.

TRAP Preview features can still change. They need `--enable-preview` to compile and run, so keep them out of production code and use them to try ideas early.

9.4 Concurrency in depth: from executors to virtual threads

Executors: tasks instead of threads The multithreading chapter created threads by hand. Real programs rarely do that. They hand tasks to an ExecutorService , which owns a pool of threads and reuses them, the way a restaurant gives orders to a fixed team of cooks instead of hiring a cook per ...

Race conditions and four fixes When two threads change the same data without coordination, the result depends on timing. That is a race condition . It is the most common reason for a test that fails "only sometimes on CI".

Visibility and the Java memory model Each CPU core keeps its own cache, and the compiler may reorder instructions. So a value written by one thread is not guaranteed to be seen by another, unless there is a happens-before relationship between them.

Concurrent collections A normal HashMap shared by threads can lose entries or, in old Java versions, even loop forever. The java.util.concurrent collections are built for sharing.

Synchronizers: latches and semaphores Synchronizers coordinate threads. A CountDownLatch lets threads wait until a count reaches zero, such as "start the tests when the database, cache and API are ready".

CompletableFuture Future.get() blocks. CompletableFuture (Java 8) lets you describe what to do when the result arrives: transform it, combine it with another, or recover from an error, without blocking a thread while you wait.

Virtual threads (Java 21) A normal (platform) thread is a real operating system thread with about a megabyte of stack, so a server can afford a few thousand.

Structured concurrency and scoped values Structured concurrency treats a group of subtasks as one unit: they start inside a block, and the block does not end until all of them are done. If one fails, the others are cancelled. No thread is left running in the background by accident.

Real world scenario: a fast smoke test Each endpoint gets its own virtual thread, a Semaphore keeps the load at 50 calls at a time, and the results go into concurrent collections. The code reads like a simple loop, finishes in a few seconds, and the report is sorted so it is the same on every run.

TRAP `volatile` fixes VISIBILITY , not ATOMICITY . `volatileCount++` still loses updates.

9.5 Inside the JVM: how a Java program really runs

The journey of a Java program The basics chapter showed the short version: javac makes bytecode and the JVM runs it. This chapter opens the JVM and looks at each stage, with real commands you can type.

Inside a class file A .class file is a small binary file with a fixed layout: a magic number, a version, a constant pool (all the names and literal values the class uses), the fields, the methods with their bytecode, and some attributes.

Class loading: delegation and lazy initialisation The JVM does not load every class at start. It loads a class the first time the code needs it. Three built in class loaders share the work, and each one asks its parent first.

The runtime memory areas A running JVM divides its memory into areas with different jobs. Every thread has its own stack of frames, one frame per method call, holding that call's local variables. All objects live on the shared heap .

Bytecode and the operand stack Bytecode instructions are one byte long, plus any operands, which is where the name comes from. The JVM is a stack machine : instructions take their inputs from a small operand stack in the current frame and push their results back onto it.

The interpreter and the JIT compiler HotSpot starts every method in the interpreter . It counts how often each method runs and how often each loop turns. When a method gets hot, the C1 compiler turns it into machine code quickly (tier 2 or tier 3, the second one also collects a profile).

Garbage collectors and object headers The garbage collection chapter explained when an object becomes garbage. The JVM ships several collectors that differ in how they clean up.

JDK Flight Recorder: see what really happened JDK Flight Recorder (JFR) is built into the JVM and records events such as garbage collections, slow locks, file and socket reads, and CPU samples, with an overhead of about one percent.

Faster startup with the AOT cache Each time a Java program starts, the JVM loads, verifies and links the same classes again. Project Leyden fixes that with an ahead of time cache .

Real world scenario: a service that misbehaves in a small container The first step is always to ask the JVM how it was started. A small health check endpoint can report it.

9.6 Annotations, reflection and modules

Annotations: labels the compiler and tools can read An annotation is a label that starts with @ . It does not change what your code does by itself. The compiler, build tools or frameworks read the label and act on it, like a "fragile" sticker on a parcel that tells the courier how to handle it.

Writing your own annotation You declare an annotation with @interface . Two meta annotations decide how it behaves. @Retention says how long it lives: only in the source (SOURCE), in the class file (CLASS, the default), or also at run time (RUNTIME).

Reflection: looking at classes while the program runs Reflection lets code examine and use classes it did not know about when it was compiled. Every loaded class has a Class object that lists its fields, methods, constructors and, for records, its components.

Dynamic proxies: how mocks and frameworks wrap your objects A dynamic proxy is an object the JVM creates at run time that implements one or more interfaces. Every call on it goes to one handler method, which can log, time, check permissions or forward the call.

The module system (Java 9) Before Java 9, a public class in any JAR could be used by any other JAR. Big programs ended up depending on each other's internals. A module fixes this.

Module import declarations (Java 25) A long list of imports at the top of a file is noise, especially for beginners and small scripts. Java 25 made module import declarations final (JEP 511).

Real world scenario: validating a sign up form The rules now live next to the data as annotations, and one generic method checks any record. This is the same idea as Jakarta Bean Validation (@NotBlank , @Min , @Max), which Spring runs automatically on request bodies.

9.7 Databases and web APIs: JDBC and HttpClient

JDBC: talking to a database Almost every real application stores data in a database. JDBC (Java Database Connectivity) is the standard API in java.sql for that. Your code uses the same interfaces whatever the database, and a driver JAR does the database specific work.

PreparedStatement and SQL injection The most famous database bug is SQL injection . When user input is glued into SQL text, a quote in the input can end the string early and change the meaning of the query. Try it in the widget first, then see the real database do it.

Transactions: all or nothing A bank transfer is two updates: take money from one account and add it to another. If the program crashes between them, money appears or disappears. A transaction groups statements so they all succeed together (commit) or all are undone (rollback).

Generated keys, batches and records Three everyday needs: getting the id the database generated for a new row, inserting many rows quickly, and turning rows into Java objects. Records are a natural fit for rows.

HttpClient: calling web APIs (Java 11) Since Java 11, the JDK has a modern HTTP client in java.net.http . It supports HTTP/1.1 and HTTP/2, synchronous and asynchronous calls, and timeouts. Each example below starts a tiny local server first with com.sun.net.httpserver , so the calls really happen.

Timeouts, async calls and HTTP/3 A call to another service can hang. Always set a connect timeout on the client and a request timeout on each request. Since Java 26 the request timeout also covers reading the response body, not only waiting for the headers.

Real world scenario: an API test suite with only the JDK The one failing check is there on purpose, to show what a failure report looks like. In bigger projects the same checks are written with REST Assured or JUnit, but they still send these requests and compare these status codes, headers and bodies.

9.8 What is new in Java: from Java 21 to Java 27

The release train, LTS and preview features Java ships a new version every six months, in March and September. Java 25 (September 2025) is the latest long term support (LTS) release, Java 26 came in March 2026 and Java 27 in September 2026.

Java 21 LTS: sequenced collections Java 21 brought virtual threads, record patterns and pattern matching for switch, each covered in its own chapter. One small change you will use daily is sequenced collections .

Java 22 and 23: native code, multi file launch, Markdown docs The foreign function and memory API (JEP 454, final in Java 22) lets Java call C libraries and work with memory outside the heap safely, replacing most JNI code. Database drivers, machine learning libraries and image tools use it to reach native code.

Java 24: quantum safe cryptography Big quantum computers could one day break RSA and elliptic curve keys. Attackers can record encrypted traffic today and decrypt it later.

Java 25 LTS: simpler programs and safer constructors Java 25 made starting with Java much simpler. Compact source files and instance main methods (JEP 512) remove the class, static and String[] args ceremony, and the new IO class reads and prints text without System.out or a Scanner.

Java 26 and 27: what is final and what to try Java 26 added HTTP/3 to HttpClient (JDBC and HTTP chapter), warns before final fields can be changed by reflection (modules chapter), removed the old Applet API (JEP 504) and let the AOT cache work with every garbage collector (JVM chapter).

Real world scenario: upgrading a service from Java 17 to Java 25 Step one is finding code that relies on JDK internals, because strong encapsulation and the removal of old APIs are what usually break upgrades.